

COMP90038 Algorithms and Complexity

Sorting with Divide-and-Conquer

Junhao Gan

Week 6 Lecture 11

Semester 1, 2026

Divide and Conquer

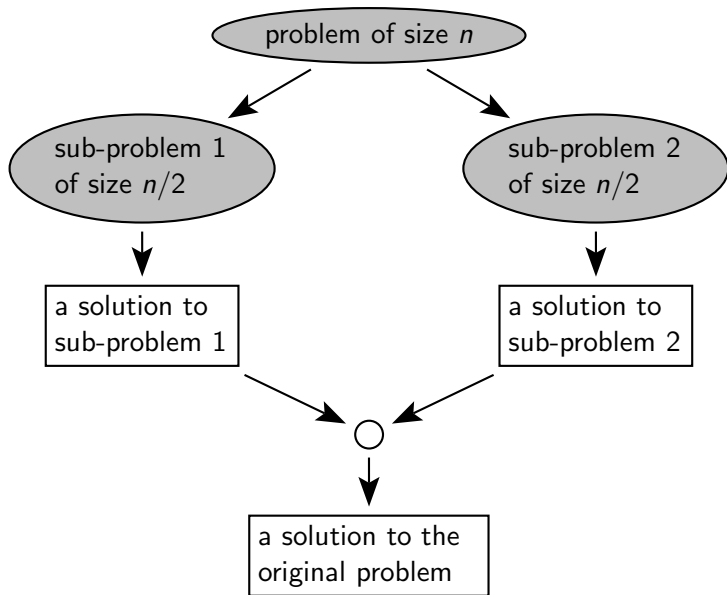
We earlier studied **recursion** as a powerful problem solving technique.

The divide-and-conquer strategy tries to make the most of this:

- 1 **Divide** the given problem instance into **smaller instances**.
- 2 Solve the smaller instances **recursively**.
- 3 **Combine** the smaller solutions to solve the original instance.

This works best when the smaller instances can be made to be of **equal** (or **roughly equal**) size.

Split-Solve-and-Join Approach



Divide-and-Conquer Algorithms

We will discuss:

- Mergesort
- Quicksort
- The Master Theorem

- Tree traversal
- Closest Pair revisited

The Sorting Problem. Given an array A of n integers, the goal is to sort them in a non-decreasing order stored in A .

In previous lectures, we have introduced some sorting algorithms, e.g., the Selection Sort, with worst-case time complexity $O(n^2)$.

Next, we introduce two efficient sorting algorithms: **the Merge Sort** and **the Quick Sort**.

The Merge Sort

The Merge Sort

Given an array A of n integers, the MergeSort algorithm works as follows:

- **The base case:** If $n = 0$ or $n = 1$, we are done and return.
- **The inductive case:**
 - **Partition** A into two (roughly) equal halves: A_{left} and A_{right} , where A_{left} consists of the first $n/2$ integers in A and A_{right} consists of the rest.
 - **Recursively** sort A_{left} .
 - **Recursively** sort A_{right} .
 - **Merge** the sorted A_{left} and A_{right} and store the result in A .

Question: How do we merge two sorted arrays?

Merging Two Sorted Lists: Example

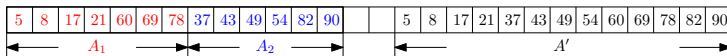
Consider two sorted arrays:

$$A_1 = \{5, 8, 17, 21, 60, 69, 78\},$$

and

$$A_2 = \{37, 42, 49, 54, 82, 90\};$$

merge them into one sorted array A' .



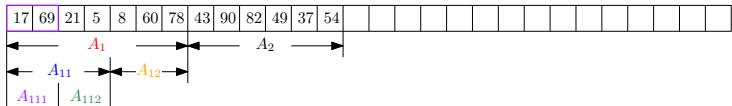
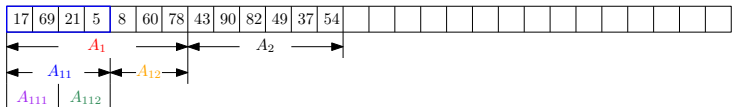
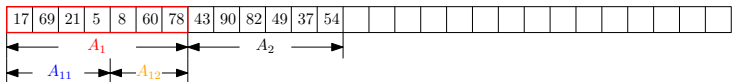
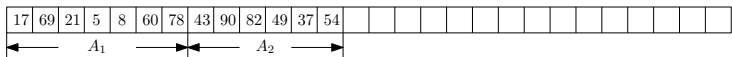
Merging Two Sorted Arrays

Given two **sorted** arrays A_1 and A_2 (each with $n/2$ integers), they can be **merged** into one sorted array by the following steps:

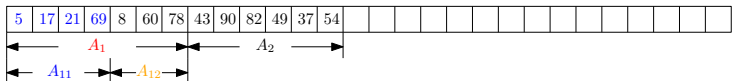
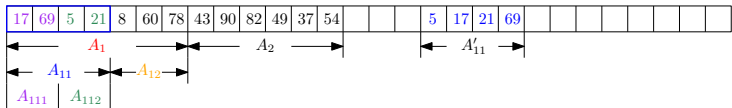
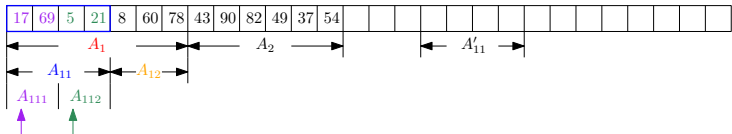
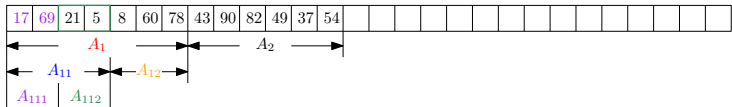
- create a new array A' of length $|A_1| + |A_2| = n$;
- maintain an index p_1 for A_1 , an index p_2 for A_2 , and an index i for A' ;
- initially $p_1 \leftarrow 0$, $p_2 \leftarrow 0$, and $i \leftarrow 0$;
- repeat the following steps until either $p_1 = |A_1|$ or $p_2 = |A_2|$:
 - if $A_1[p_1] \leq A_2[p_2]$,
 - $A'[i] \leftarrow A_1[p_1]$ and $p_1 \leftarrow p_1 + 1$;
 - otherwise,
 - $A'[i] \leftarrow A_2[p_2]$ and $p_2 \leftarrow p_2 + 1$;
 - $i \leftarrow i + 1$;
- if $p_1 = |A_1|$, **append** all the rest integers in A_2 to A' ;
- otherwise, **append** all the rest integers in A_1 to A' .

The Merge Sort: Example

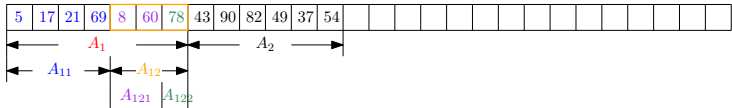
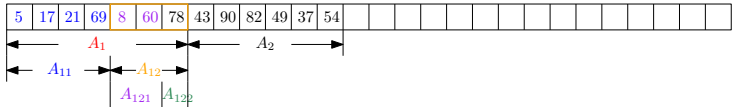
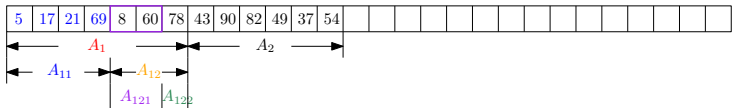
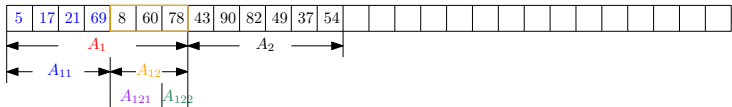
Let us go back to the MergeSort and consider the following running example.



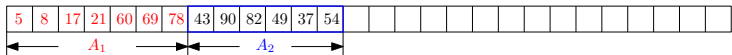
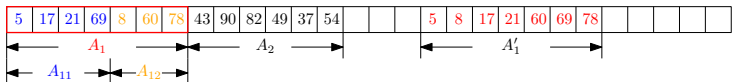
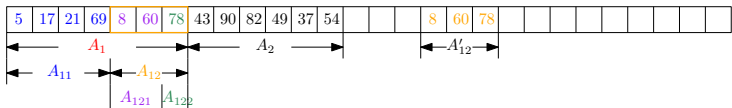
The Merge Sort: Example



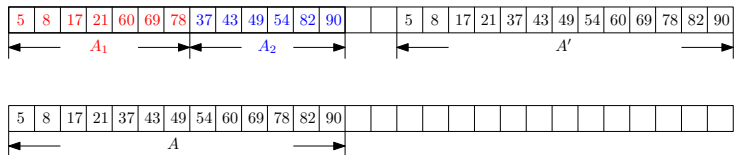
The Merge Sort: Example



The Merge Sort: Example



The Merge Sort: Example



The Merge Sort: Cost Analysis

Define $C(n)$ as the **worst-case** cost of the MergeSort sorting n integers. Then we have the following recurrence:

$$\begin{aligned} C(n) &= O(1), & \text{for } n = 0 \text{ or } n = 1 \\ C(n) &= 2 \cdot C(n/2) + O(n), & \text{for } n > 1 \end{aligned}$$

Solving the above recurrence by expansion, we have $C(n) \in O(n \cdot \log n)$.

Therefore, the worst-case time complexity of the MergeSort is bounded by $O(n \cdot \log n)$.

A Comparison Lower Bound of the Sorting Problem

In the **Comparison Computation Model**, an algorithm is *only* allowed to learn which is greater, smaller or equal between any two elements by **comparing** them.

In this model, **any correct** comparison-based algorithm has to perform $\Omega(n \cdot \log n)$ comparisons to sort n elements.

Therefore, the MergeSort is indeed an **optimal** comparison-based sorting algorithm.

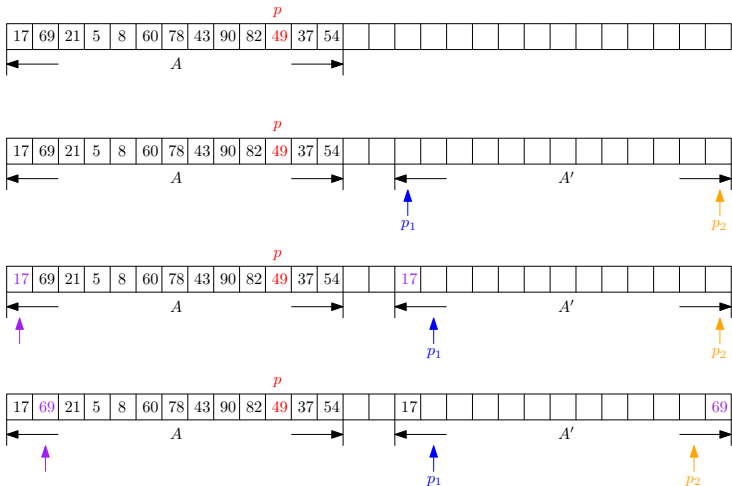
The Quick Sort

The Quick Sort

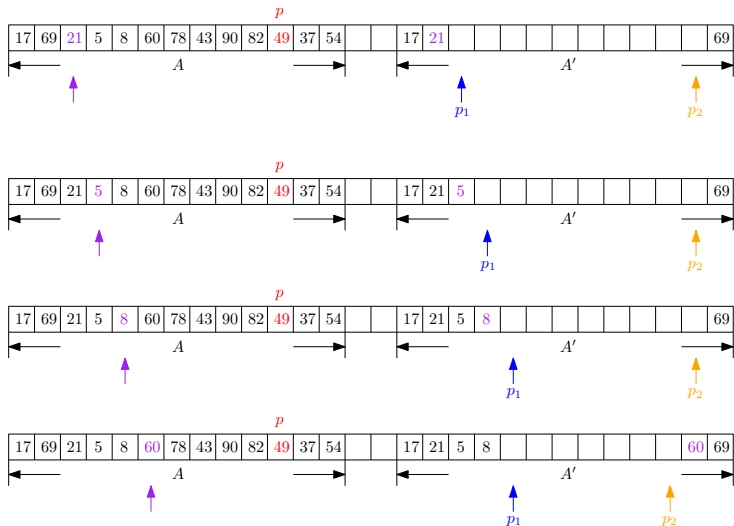
Given an array A of n integers, the QuickSort algorithm works as follows:

- **The base case:** If $n = 0$ or $n = 1$, we are done and return.
- **The inductive case:**
 - Pick a **pivot** p from A ;
 - **Partition** the integers in A such that:
 - All the integers $\leq p$ are positioned **before** p in A ;
 - All the integers $> p$ are positioned **after** p in A ;
 - **Recursively** sort A_1 , the part of A before p ;
 - **Recursively** sort A_2 , the part of A after p ;

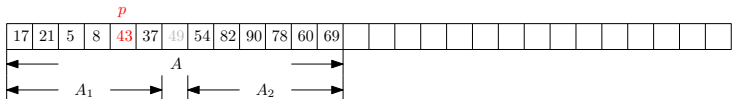
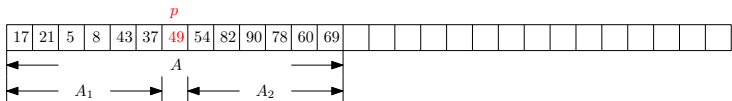
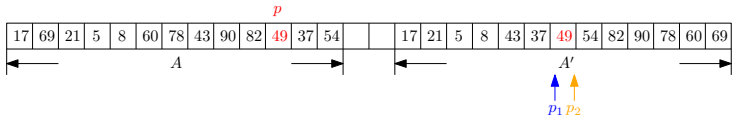
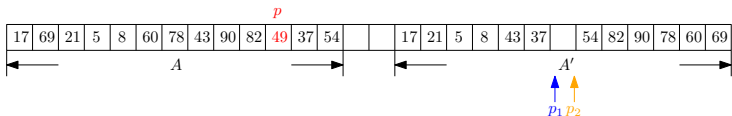
The Quick Sort: Example



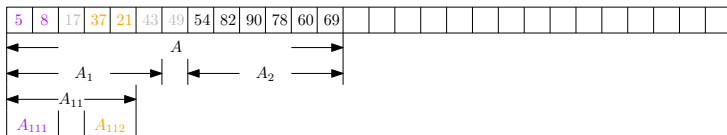
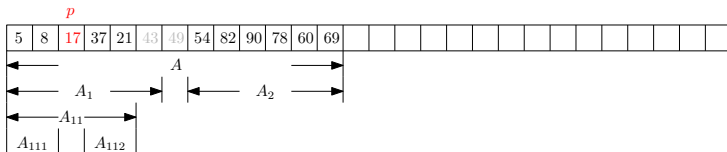
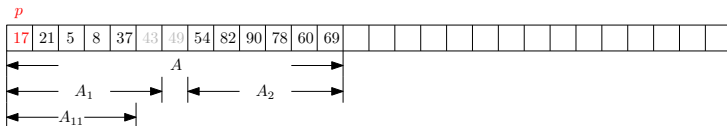
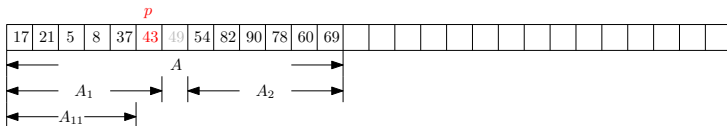
The Quick Sort: Example



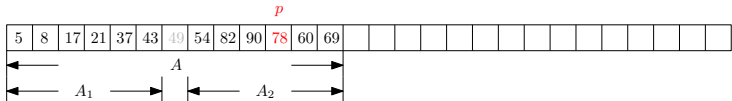
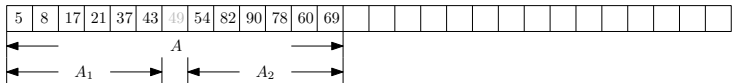
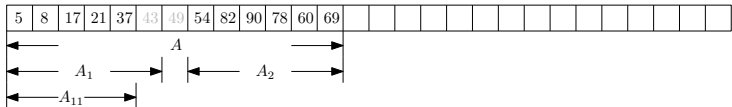
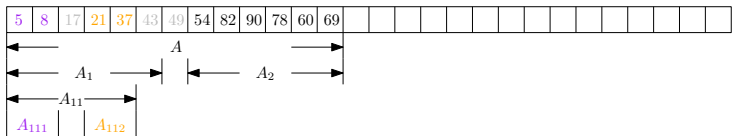
The Quick Sort: Example



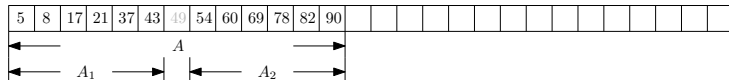
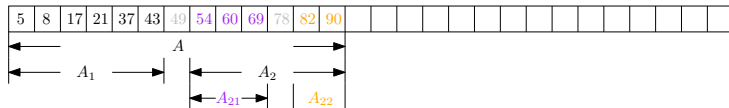
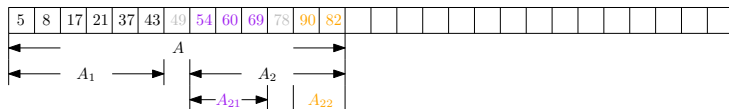
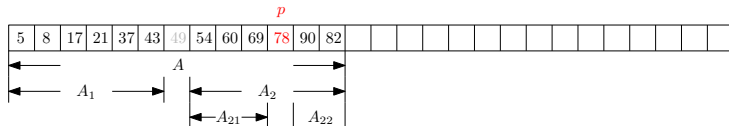
The Quick Sort: Example



The Quick Sort: Example



The Quick Sort: Example



The Quick Sort: Example



Question: What is the worst-case cost of the QuickSort?

The Quick Sort: Cost Analysis

Similar to the k -**Selection Problem**, in the worst case, it is possible to pick a **very bad** pivot such that either A_1 or A_2 is empty.

In this case, there can be n iterations, each of which takes $O(n)$ time.

Therefore, the overall worst-case time complexity of QuickSort is bounded by $O(n^2)$.

Question: Any idea of improving the algorithm?

The Randomized Quick Sort

Apply the same randomized trick as we did in the Randomized QuickSelection algorithm:

- When we pick a pivot, we *uniformly at random* pick an element p in the current array A as a pivot;
- Partition A with p ;
- If both $|A_1| \leq 2/3 \cdot |A|$ and $|A_2| \leq 2/3 \cdot |A|$ hold,
 - accept p and continue the QuickSort;
- Otherwise, reject p and repeat from the first step;

The Randomized Quick Sort: Cost Analysis

The probability of picking a good pivot is at least $1/3$.

In expectation, with **at most 3 trials**, we can pick a good pivot.

Considering the recursion tree, it can be verified that, at each level, the total dividing cost of all recursions there is bounded by $O(n)$ in expectation.

As there are at most $O(\log n)$ levels in the recursion tree, the overall **expected time complexity** of the Randomized QuickSort is bounded by $O(n \cdot \log n)$.

The Randomized Quick Sort: Cost Analysis (II)

Now, let us consider a different way of analysis.

Consider the following **expected cost** recurrence

$$\begin{aligned} C(n) &= O(1), & \text{for } n = 0 \text{ or } n = 1 \\ C(n) &\leq 2 \cdot C\left(\frac{2}{3} \cdot n\right) + O(n), & \text{for } n > 1 \end{aligned}$$

Unfortunately, solving this recurrence does not give us a $O(n \log n)$; instead, it turns out a bound of $O(n^{\log_{\frac{3}{2}} 2})$ which is worse than $O(n \log n)$. (You may solve this recurrence by the expansion trick or the Master Theorem as introduced below.)

The Randomized Quick Sort: Cost Analysis (II)

The reason for this is that in the above recurrence, the cost is *overestimated* by $2 \cdot C(\frac{2}{3} \cdot n)$ which is the total size of the two “smaller” problems is larger than the original problem size n .

However, indeed, the total size of the two “smaller” problems should be n .

More specifically, they must have sizes $\alpha \cdot n$ and $(1 - \alpha) \cdot n$, for some $\alpha \in [\frac{1}{3}, \frac{2}{3}]$.

$$\begin{aligned} C(n) &= O(1), & \text{for } n = 0 \text{ or } n = 1 \\ C(n) &\leq \max_{\alpha \in [\frac{1}{3}, \frac{2}{3}]} \{C(\alpha \cdot n) + C((1 - \alpha) \cdot n)\} + O(n), & \text{for } n > 1 \end{aligned}$$

The Master Theorem

Divide-and-Conquer Recurrences

What is the time required to solve a problem of size n by divide-and-conquer?

For the general case, assume we split the problem into b instances (each of size n/b), a of which need to be solved:

$$T(n) = aT(n/b) + f(n)$$

where $f(n)$ expresses the cost for dividing a problem into b sub-problems and combining the a results.

(A very common case is $T(n) = 2 \cdot T(n/2) + n$.)

How to find closed forms for these recurrences?

The Master Theorem

(A proof is in Levitin's Appendix B.)

For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d)$, $d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with $T(1) = c$) has solutions, and

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

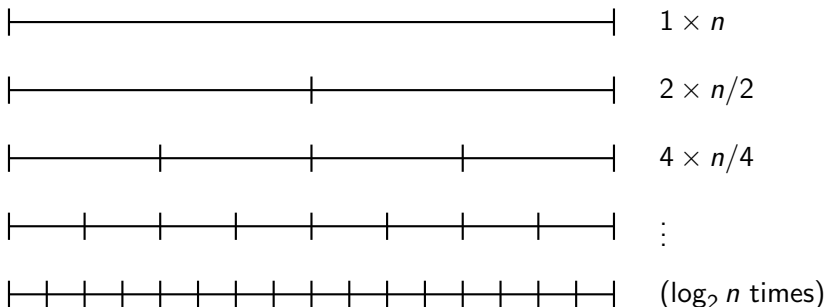
Note that we also allow a to be greater than b .

The Master Theorem also holds for big- O and big- Ω .

Master Theorem: Example 1

$$T(n) = aT(n/b) + \Theta(n^d) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

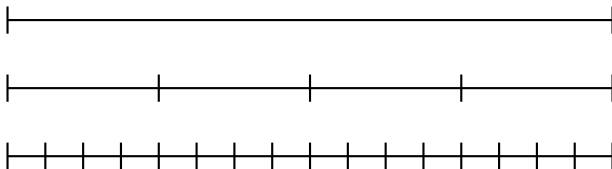
$$T(n) = 2T(n/2) + n \qquad a = 2, b = 2, d = 1$$



Master Theorem: Example 2

$$T(n) = aT(n/b) + \Theta(n^d) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

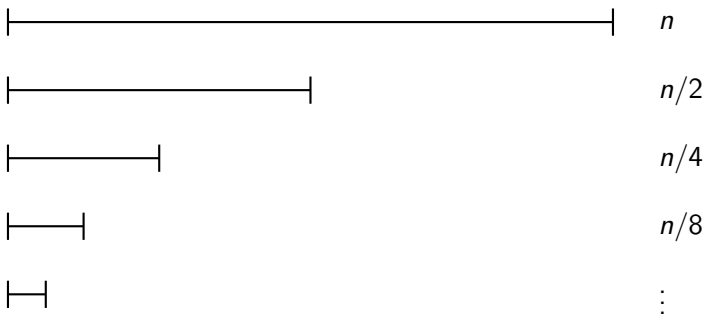
$$T(n) = 4T(n/4) + n \qquad a = 4, b = 4, d = 1$$



Master Theorem: Example 3

$$T(n) = aT(n/b) + \Theta(n^d) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

$$T(n) = T(n/2) + n \qquad a = 1, b = 2, d = 1$$



Master Theorem: Example 4

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

$$T(n) = 2T(n/2) + n^2$$

$$a = 2, b = 2, d = 2$$

In these last two examples $a < b^d$,
and we simply get n^d .

